



FEU Institute of Technology
COLLEGE OF ENGINEERING • COLLEGE OF COMPUTER STUDIES

College of Computer Studies

OBJECT ORIENTED PROGRAMMING
(CCS0070L)

SCHOOL ORGANIZATION EVENT MANAGER

Submitted by:

Elecanal, Erwin Cyrus John T. | BSCSAI-TN22
Submitted to: Doc. Tejueco

I. INTRODUCTION

A school event can look simple on a poster, but someone still has to keep track of the date, venue, leader, age limit, available slots, and people who signed up. When these details are kept in separate notes, it is easy to lose track of the current registration count. School Organization Event Manager puts those tasks in one Java console program. Managers maintain events, while attendees can find and join them through their own accounts.

This problem is a useful fit for a small console project because the two roles have different jobs but share basic account information. The application is a prototype. It keeps data only while it is open and does not include a database, email notifications, or payments.

II. DESCRIPTION OF THE PROJECT

The first menu asks the user to choose Attendee or Event Manager. A manager can create an account, log in, add an event, change an event they own, cancel it, and see both event details and registrants. An attendee can create an account, log in, browse events, register, cancel a registration, and check which events they are currently joining. Each event shows its title, description, schedule, venue, leader, minimum age, capacity, status, and registration count.

The program checks each registration before accepting it. It refuses a duplicate sign-up, a full event, a canceled event, an event that has already started, or an attendee who will be underage on the event date. The account name is Lastname,Firstname. A separate registration label adds the event title, so a student can join more than one event without creating another account.

III. OBJECTIVES

- Give managers and attendees separate accounts and menus in a working Java console program.
- Let managers maintain their own events and see who registered.
- Check age, capacity, duplicate entries, and event status before accepting a registration.
- Show a registration count based on active records, including after a cancellation.
- Use classes, inheritance, encapsulation, polymorphism, an abstract class, and an interface where they fit the design.
- Test both successful actions and common mistakes, including empty input and boundary values.

IV. SIGNIFICANCE OF THE STUDY

The program gives an event manager one place to check an event and its sign-ups. An attendee can see the age rule and number of slots before trying to register. The project also shows how a real task can be split into manageable parts: account and event classes hold information, service classes handle actions, and a policy class checks registration rules. Because this is a classroom prototype, passwords are only sample data, nothing is saved after the program closes, and one person uses the console at a time.

V. PROJECT DETAILS

The project has 13 Java source files. Accounts, events, and registrations are kept in lists during a program run. The UML diagram shows how the classes connect and which classes are responsible for each part of the work.

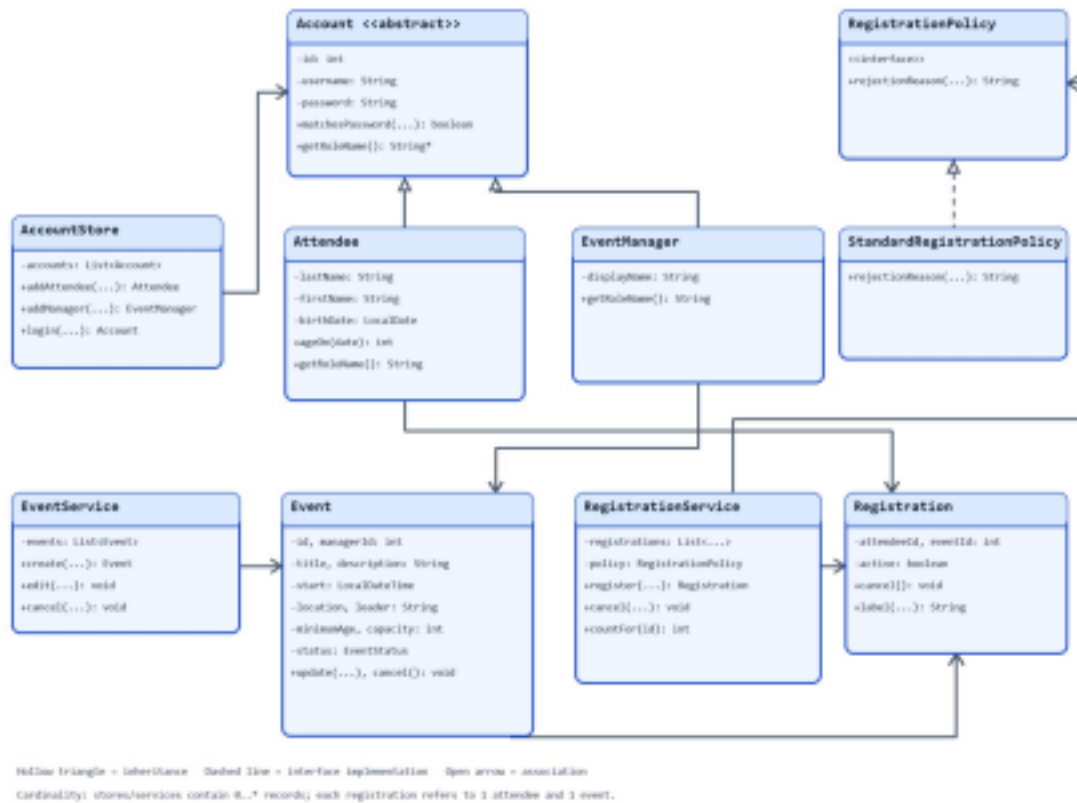


Figure 1. UML class diagram of the account, event, registration, service, and policy classes.

Table of Functions

Function	Scope	Purpose
Main.run and roleMenu	private	Repeat and route console menus
Main.required and readInt	private	Validate required text and integer ranges
AccountStore.login	public	Check credentials and role
EventService.create, edit, cancel	public	Maintain manager-owned events
RegistrationService.register, cancel	public	Change active registrations
RegistrationService.countFor	public	Count active registrations for an event
StandardRegistrationPolicy.rejectionReason	public	Apply registration eligibility rules

Table of Classes

Class or interface	Key access	Responsibility
Account; Attendee; EventManager	private fields; public methods	Shared login data and role-specific account behavior
Event	private fields; public methods	Event details, status, and validated updates
Registration	private fields; public methods	Link attendee and event; track active state

Class or interface	Key access	Responsibility
AccountStore; EventService	private lists; public methods	Manage accounts and manager-owned events
RegistrationService	private list and policy; public methods	Register, cancel, list, and count
RegistrationPolicy; StandardRegistrationPolicy	public interface method	Define and apply acceptance conditions
Main; SelfTest; EventStatus	mixed	Console control, verification, and status values

Table of Data Structures and ADT

Identifier	Structure and operations	Purpose
accounts	ArrayList<Account>; add, iterate, search	Hold attendee and manager accounts
events	ArrayList<Event>; add, iterate, search	Hold events and find manager-owned records
registrations	ArrayList<Registration>; add, iterate, filter	Track active participation and counts

Table of Algorithms

Algorithm	Location	Purpose
Age on event date	Attendee.ageOn; registration policy	Compare attendee age with each event's rule
Eligibility scan	StandardRegistrationPolicy	Reject duplicate entries and full events
Active count	RegistrationService.countFor	Derive displayed attendance from records
Input retry	Main.required; Main.readInt	Repeat prompts until a permitted value is entered

Division of Labor

Work	Responsible	Notes
Project decisions, code review, testing, and presentation	[Student name]	Explain and check the final work
Ideas, explanations, debugging suggestions, and draft text	ChatGPT assistance	Suggestions checked against the program

VI.SCREEN OUTPUT

These captures show what happens when a manager creates an event, an attendee signs up and cancels, and the program handles an invalid choice. The last image shows the test run in NetBeans.

```
Java console capture: manager creates and views an event

Choose: Username: Password: Welcome, clubadmin (Event Manager).

Manager: 1 Create event  2 Edit event  3 Cancel event  4 My events  5 Registrants  0 Log out
Choose: Title: Description: Start (yyyy-MM-dd HH:mm): Location: Leader: Minimum age (0-120): Capacity

Manager: 1 Create event  2 Edit event  3 Cancel event  4 My events  5 Registrants  0 Log out
Choose:
#1 School Fair [SCHEDULED]
  Student club exhibits
  When: 2027-05-20 09:00 | Where: Main Hall
  Leader: Club Lead | Minimum age: 18
  Registered: 0/2
```

Figure 2. Manager creates and views School Fair with zero of two places filled.

```
Java console capture: attendee registers and cancels

Choose: Username: Password: Welcome, Reyes,Ana (Attendee).

Attendee: 1 Browse events  2 Register  3 Cancel registration  4 My registrations  0 Log out
Choose:
#1 School Fair [SCHEDULED]
  Student club exhibits
  When: 2027-05-20 09:00 | Where: Main Hall
  Leader: Club Lead | Minimum age: 18
  Registered: 0/2

Attendee: 1 Browse events  2 Register  3 Cancel registration  4 My registrations  0 Log out
Choose: Event ID: Registered as Reyes,Ana-School Fair.

Attendee: 1 Browse events  2 Register  3 Cancel registration  4 My registrations  0 Log out
Choose: Event ID: Error: Already registered for this event.

Attendee: 1 Browse events  2 Register  3 Cancel registration  4 My registrations  0 Log out
Choose: #1 Reyes,Ana-School Fair [SCHEDULED]

Attendee: 1 Browse events  2 Register  3 Cancel registration  4 My registrations  0 Log out
Choose: Event ID: Registration canceled.

Attendee: 1 Browse events  2 Register  3 Cancel registration  4 My registrations  0 Log out
Choose: No active registrations.
```

Figure 3. Attendee registers, receives a duplicate warning, and cancels the registration.

1	Attendee account/login	Reyes; Ana; 2005-05- 01; secret1	Username and login succeed	Username and login succeeded	Pass
2	Role polymorphism	Attendee and EventManager objects	Distinct role names via Account references	Attendee and Event Manager returned	Pass
3	Duplicate username	reyes,ana after Reyes,Ana	Reject case-insensitive duplicate	Duplicate rejected	Pass
4	Incorrect password	lead; wrong	Login fails	Login returned no account	Pass
5	Minimum capacity	Age 0; capacity 1	Event created with boundary values	Event created with boundary values	Pass
6	Empty title	Whitespace title	Reject event	Event rejected	Pass
7	Invalid capacity	Capacity 0	Reject event	Event rejected	Pass
8	Past event	2025-01-01 09:00 with fixed 2026 clock	Reject past start	Past start rejected	Pass
9	Minimum age boundary	Age exactly 18 on event day; minimum 18	Accept registration	Registration accepted	Pass
10	Below minimum age	Age 17 on event day; minimum 18	Reject registration	Registration rejected	Pass
11	Duplicate registration	Same attendee registers twice	Second attempt rejected and count stays 1	Second attempt rejected and count stayed 1	Pass
12	Capacity reached	Capacity 1; second attendee	Reject second attendee	Second attendee rejected	Pass
13	Cancel and repeat	Register; cancel; register again	Count changes 1 to 0 to 1	Count changed 1 to 0 to 1	Pass
14	Registration label	Reyes Ana joins School Fair	Reyes,Ana-School Fair	Correct label returned	Pass
15	Canceled event	Attempt registration after manager cancels	Reject registration	Registration rejected	Pass
16	Started event	Attempt registration after start	Reject registration	Registration rejected	Pass
17	Event ownership	Other manager edits event	Reject unauthorized edit	Edit rejected	Pass
18	Capacity edit boundary	Two registrations then capacity 1	Reject reduction	Reduction rejected	Pass
19	Eligibility change after registration	Change event start and minimum age	Reject change	Change rejected	Pass
20	Multiple events	One attendee joins two events	Two registrations under one account	Two registrations under one account	Pass
21	Incorrect menu selection	9 at main menu	Show invalid choice and continue	Invalid choice shown and main menu repeated	Pass
22	Empty required input	Blank manager username then lead	Show error and retry username	Error shown and account created after retry	Pass

#	Feature	Input	Expected	Actual	Result
23	Upper boundary	Minimum age 120; capacity 1000	Accept valid upper and reasonable capacity values	Event created with both values	Pass
24	Above maximum age rule	Minimum age 121	Reject event	Event rejected	Pass

The automated run finished with 24 passed and 0 failed. The test code is in SelfTest.java, and the table is also saved as test-cases.csv.

VIII. SOURCE CODE

All 13 Java files are in src/main/java/com/mycompany/event_organizer. These short excerpts show three important parts of the program: how a blank answer is handled, how eligibility is checked, and how the count is calculated.

Input validation in Main.java

```
// Repeats a required text prompt until the user enters content.
private String required(String string) {
    String string2;
    while ((string2 = this.read(string)).isBlank()) {
        System.out.println("This field cannot be empty.");
    }
    return string2;
}
```

Eligibility policy in StandardRegistrationPolicy.java

```
// A started or canceled event cannot accept a new registration.
if (event.getStatus() == EventStatus.CANCELED) return "Event is canceled.";
if (!event.getStart().isAfter(localDateTime)) return "Event has already
started."; // Compare age on the event day, not on the day of sign-up.
if (attendee.ageOn(event.getStart().toLocalDate()) < event.getMinimumAge())
    return "Attendee does not meet the event's minimum age.";
```

Active count in RegistrationService.java

```
// Count active records for this event; canceled records do not count.
public int countFor(int n) {
    int n2 = 0;
    for (Registration registration : this.registrations) {
        if (!registration.isActive() || registration.getEventId() != n) continue;
        ++n2;
    }
    return n2;
}
```

Account is abstract; Attendee and EventManager extend it and override getRoleName(). RegistrationService depends on the RegistrationPolicy interface, implemented by StandardRegistrationPolicy. Private fields and validated methods control changes to accounts, events, and registrations.

IX. OBJECT ORIENTED DESIGN

Classes and relationships

Attendee and EventManager are kinds of Account, so they use inheritance. A manager owns zero or more Events. An attendee can make zero or more Registrations, and each Registration links to one Event. AccountStore, EventService, and RegistrationService each manage their own collection. These relationships are shown with multiplicities in the UML diagram.

Encapsulation

Account, Event, and Registration keep state in private fields. Constructors and methods validate changes; Event.update requires details, age 0 to 120, and capacity of at least one.

Inheritance, overriding, and polymorphism

Attendee and EventManager call super(...) to initialize shared Account fields and override getRoleName(). AccountStore holds both as Account references; the actual subclass determines which role-name method runs.

Abstraction and interface

Account is abstract because a generic account has no role identity. RegistrationPolicy declares rejectionReason; StandardRegistrationPolicy applies the current event rules. RegistrationService depends on that interface, separating eligibility from record management.

X. DEVELOPMENT PROCESS AND AI EVALUATION

Brainstorming and refinement

The idea started as an event list that managers could edit. Adding attendee accounts raised practical questions: What if someone signs up twice? What happens when an event is full? How should an age limit be checked? Working through those questions led to registration and cancellation rules. It also showed why the event name belongs on a registration label rather than in the attendee's account name.

Suggestions considered

Suggestion	Evaluation	Decision
Use Lastname,Firstname-Event as the account name	One person joining two events would need two usernames or a misleading suffix.	Modified: keep one account and add the event title to each registration label.
Store a numeric attendance count on Event	A second stored count could disagree with cancellations and repeat registrations.	Rejected: count active Registration records when displaying an event.
Use inheritance for account roles and an interface for registration rules	Both roles truly are Accounts; eligibility rules can vary separately.	Accepted: abstract Account, two subclasses, and RegistrationPolicy.

Problems encountered and improvements

One early naming idea would have made the same attendee use a different account name for each event. Separating the account name from the registration label solved that problem. Date tests also showed why age must be checked on the event day, especially around a birthday. The program now protects existing sign-ups when a manager edits capacity or eligibility. After source recovery caused compiler errors, the affected files were repaired and tested again.

How ChatGPT helped

ChatGPT was used to discuss the class design, suggest edge cases, explain errors, and help draft this report. Its suggestions were checked against the actual program. The source compiled, and the 24 tests passed. The complete conversation should be submitted separately as the record of those discussions.

Personal learning

[Student: write your own learning reflection here, using one code example you can explain during the defense.]

XI. CONCLUSION

School Organization Event Manager now lets managers keep event information together and lets attendees register through their own accounts. Its rules stop duplicate, underage, late, and over-capacity registrations, while the displayed count comes from active records. All 24 documented tests passed in NetBeans. The application works as a local classroom prototype; saving data between runs and improving account security would be sensible next steps.